

3D Embedded Scanner

Yuval Rubin
David Khutsishvili

Contents

1	Introduction	2
2	System Design	2
2.1	Architecture Overview	2
2.2	Microcontroller	3
2.3	Sensors	3
2.4	Actuators	3
2.5	Mechanical Structure	3
3	Implementation	4
3.1	Sensor Initialization and the XSHUT Problem	4
3.2	Robust Distance Reads	4
3.3	Serpentine Scanning and Object Detection	4
3.4	Host-Side Capture	5
3.5	Point Cloud Reconstruction	5
3.6	Problems Encountered – Summary	5
4	Testing and Results	6
4.1	Subsystem Bring-Up Tests	6
4.2	End-to-End Scans	6
4.3	Observations	7
5	Conclusion	7

1 Introduction

The goal of this project was to build a 3D scanner from readily available components: an Arduino microcontroller, two time-of-flight (ToF) distance sensors, and a set of servos. Rather than moving the sensors around the object, the sensors were fixed in place on two pillars and the object itself was moved: it sat on a plate that rotated a full turn at a fixed elevation, and was then stepped down before rotating again. Sweeping the object through the fixed sensors' line of sight this way traced out a ring of distance samples over the surface of the object, in cylindrical coordinates (θ, h, r) .

The core of the project was the embedded system: an Arduino sketch that sequenced the sensors and actuators and drove the entire scanning process, streaming raw measurements over serial as they were taken. A small Python toolchain on the host computer supported this by capturing that serial stream to a file and reconstructing a 3D point cloud from it, which was rendered for inspection.

2 System Design

2.1 Architecture Overview

The system was split into two halves that only communicated over a serial (USB) link: the embedded scanner, which physically performed the scan, and a host-side Python toolchain, which captured and visualized the resulting data. Figure 1 shows the overall data and control flow.

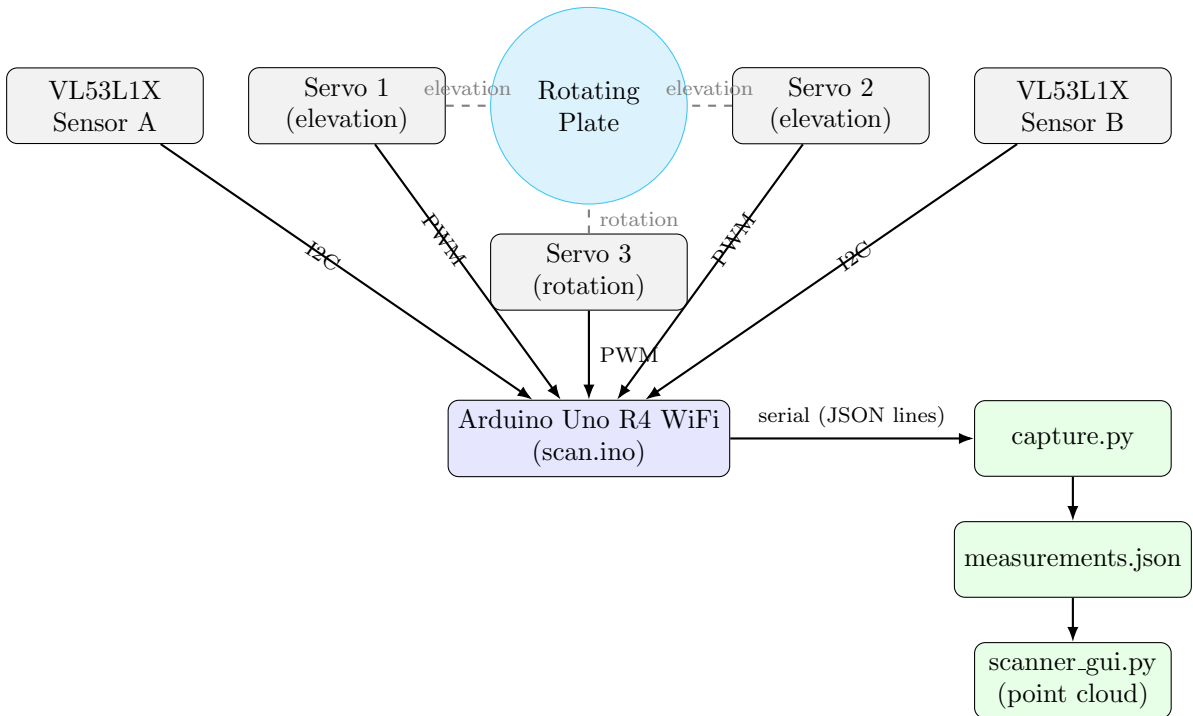


Figure 1: System architecture, laid out to match the physical bench arrangement: sensor A, servo 1, the rotating plate, servo 2, and sensor B sit side by side, with servo 3 driving the plate's rotation from below. The Arduino drove all five devices directly and streamed JSON measurement lines over serial; the host captured them to a file and turned them into a point cloud.

2.2 Microcontroller

The scanner was controlled by an **Arduino Uno R4 WiFi**. Only its standard digital I/O, PWM, and I2C peripherals were used (WiFi was not used by the project). It was responsible for:

- bringing up and reading both distance sensors over I2C,
- driving the three servos over PWM,
- sequencing the scan (elevation rings, rotation steps),
- streaming measurements to the host as line-delimited JSON over the serial (USB) connection.

2.3 Sensors

Two **Adafruit VL53L1X** time-of-flight distance sensors were mounted a fixed distance apart (`SENSOR_DIST`) on two static pillars, aimed inward toward the object. Using two sensors instead of one let the system sample two points on the object's surface per rotation step, and (together with the known sensor geometry) helped disambiguate the object's shape.

Both sensors were identical modules and booted up at the same default I2C address (`0x29`), which would have made them indistinguishable on a shared I2C bus. Each board's **XSHUT** pin (active-low shutdown) was used to bring the sensors up one at a time so one of them could be moved to a different address before the other was released onto the bus (see Section 3.1).

2.4 Actuators

Three **MG996R** servos drove the mechanism:

- `servo1` and `servo2` moved in tandem to raise/lower the elevation of the object plate;
- `servo3` rotated the plate the object was mounted on.

All three were standard PWM servos, attached with an extended pulse range (`MIN_PULSE_US/MAX_PULSE_US` of 500–2500 μ s) to make use of their full mechanical travel.

2.5 Mechanical Structure

Unlike a design where the sensors sweep around a stationary object, this scanner kept the two ToF sensors **fixed** on pillars. The object being scanned sat on a plate that:

1. started at maximum elevation,
2. rotated a full turn while the sensors sampled distances at fixed angular increments (an “elevation ring”),
3. was lowered by a small step,
4. repeated, rotating the opposite direction (serpentine motion) to avoid snapping the plate back to 0° between rings.

This continued until an entire ring returned no detections, at which point the object was assumed to have been scanned from top to bottom.

3 Implementation

3.1 Sensor Initialization and the XSHUT Problem

The first problem encountered when wiring up two identical VL53L1X boards was that they could not be told apart on the I2C bus: both powered up listening on address 0x29. The `init_sensors()` routine solved this using each sensor’s XSHUT pin:

Listing 1: Sequenced sensor bring-up (excerpt from scan.ino)

```
1 digitalWrite(XSHUT_A_PIN, LOW);
2 digitalWrite(XSHUT_B_PIN, LOW);
3 delay(10);
4 Wire.begin();
5
6 // Sensor A is woken and immediately reassigned to 0x30
7 vl53_a.begin(SENSOR_A_ADDR, &Wire);
8
9 // 0x29 is now free; sensor B is woken and stays at the default
10 vl53_b.begin(SENSOR_B_ADDR, &Wire);
```

Both sensors were held in shutdown first, guaranteeing neither responded on the bus. Sensor A was then released and moved to 0x30; only once it was safely off the default address was sensor B released, so it could keep the default address 0x29 without a collision.

3.2 Robust Distance Reads

A second issue observed during bring-up was that a sensor occasionally failed to report a new reading before it was polled, or intermittently returned an invalid value. `read_distance()` handled both cases:

- it polled `dataReady()` for up to `DATA_READY_TIMEOUT_MS` before giving up on that attempt, and retried up to `READ_RETRY_COUNT` times;
- a negative distance was treated as “out of range” and replaced with a large sentinel value (`OUT_OF_RANGE_MM`) rather than corrupting the average.

On top of this, `measure()` took `MEASURE_COUNT` samples per angular step and averaged them, which reduced the effect of sensor noise on each recorded point.

3.3 Serpentine Scanning and Object Detection

The scanning loop (`scanning_routine()`) stepped the plate through a full rotation at each elevation, reversing the rotation direction (`d_rotational_angle *= -1`) after every ring instead of resetting the plate back to 0°. This avoided wasting time and mechanical wear on an unnecessary reset step between rings.

A measurement was only reported (and printed as a JSON line) when both sensors detected the object closer than `DETECTED_THRESH_MM` (Equation 1); this filtered out background readings when the sensors were looking past the object at the empty scanner frame. If an entire ring produced no detections at all, the scan was considered complete and stopped early rather than continuing to sweep empty air:

$$\text{detected} = (\overline{d_A} < T) \wedge (\overline{d_B} < T), \quad T = \text{DETECTED_THRESH_MM} \quad (1)$$

Each accepted measurement was serialized as a single line of JSON:

Listing 2: Example measurement line

```
1 {"theta":42,"h":12.960,"dist_a":88,"dist_b":91}
```

with the whole scan bracketed by `SCAN_START` / `SCAN_END` markers, so a host-side listener could unambiguously tell when a scan began and ended on an otherwise plain serial stream.

3.4 Host-Side Capture

`capture.py` read the Arduino’s serial port line by line, waited for `SCAN_START`, and parsed every subsequent line as JSON until it saw `SCAN_END`. A line that failed to parse (for example, a `WARN:` log message emitted by the sketch when a sensor read timed out) was printed for visibility but did not abort the capture – this proved important during development, since the sensors would occasionally log a warning without it indicating a fatal problem. The resulting list of measurements was written out as a single JSON file. Capture could also be interrupted with `Ctrl-C`, in which case whatever had been captured so far was still saved.

3.5 Point Cloud Reconstruction

`scanner_gui.py` turned the flat list of (θ, h, d_A, d_B) measurements into 3D points. Each sensor was offset from the plate’s rotation axis by half of `SENSOR_DIST`, looking inward, so a measured distance was first converted to a point in the plate’s own frame and then rotated by θ about the vertical axis:

$$p_A = R_z(\theta) \cdot \left(\frac{\text{SENSOR_DIST}}{2} - d_A, 0, h \right)^\top$$

$$p_B = R_z(\theta) \cdot \left(-\frac{\text{SENSOR_DIST}}{2} + d_B, 0, h \right)^\top$$

The resulting points from every measurement were collected into a single point cloud and rendered with PyVista (Section 4).

3.6 Problems Encountered – Summary

Problem	Solution
Both ToF sensors shared the same default I2C address	Sequenced XSHUT bring-up: held both in reset, woke sensor A and reassigned its address, then woke sensor B onto the freed default address
Occasional sensor read timeouts / invalid readings	Bounded retry loop with a data-ready timeout, treating negative readings as “out of range” instead of propagating bad data
Noisy individual distance samples	Averaged <code>MEASURE_COUNT</code> reads per angular step before recording a measurement
Wasted motion resetting rotation between elevation rings	Serpentine scan pattern: alternated rotation direction each ring
Knowing when the object had been fully scanned	Stopped the scan once a whole ring returned no detections above threshold
Non-measurement log lines mixed into the serial stream	<code>capture.py</code> treated a JSON-parse failure as an informational log line rather than a fatal error

Table 1: Problems encountered during development and how each was addressed.

4 Testing and Results

4.1 Subsystem Bring-Up Tests

Before integrating the full scan routine, each subsystem was validated in isolation using standalone sketches under `src/scanner/hardware_test/`:

- **zero_servo** – confirmed a single servo could be driven to its zero and maximum positions reliably.
- **reset_elevation_gear** – verified the elevation gear train returned to a consistent zero/reset position.
- **elevation_control** – swept `servo1/servo2` together through their full elevation range to confirm they stayed in sync.
- **rotation_control** – stepped `servo3` through a full rotation to validate the plate’s rotation increments.
- **laser_test** – brought up both VL53L1X sensors using the XSHUT sequencing described in Section 3.1 and printed live distance readings from each, confirming both sensors could be addressed and read independently on the shared I2C bus.

Only after each of these passed independently was the combined `scan.ino` sketch built and run.

4.2 End-to-End Scans

Two full end-to-end scans were captured and were included in the repository under `scans/`:

- `green_apple.json` – a scan of a small round object.
- `pear_real.json` – a scan of an irregularly-shaped object, which produced a substantially larger number of measurements due to its larger vertical extent.

Each file was captured with `capture.py` directly from the running sketch’s serial output, then rendered into a point cloud with `scanner_gui.py`.

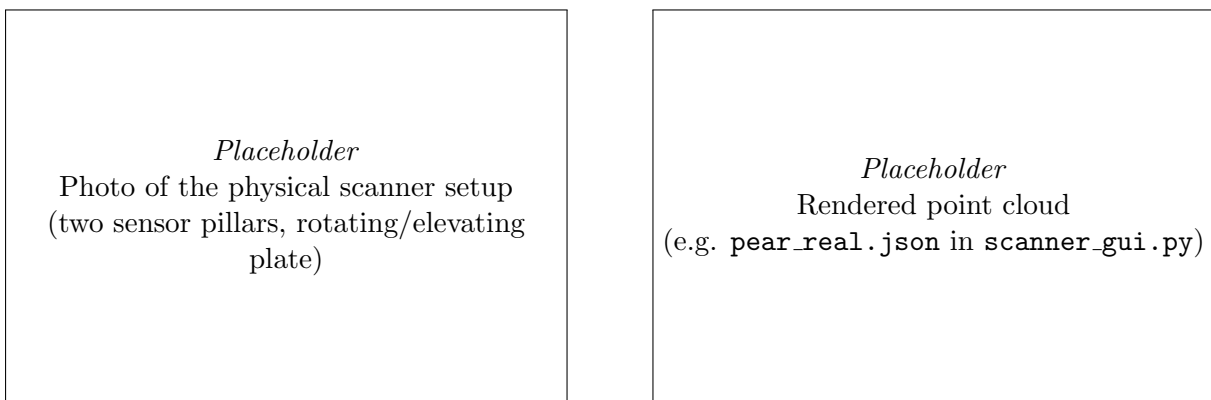


Figure 2: Physical setup (left) and an example reconstructed point cloud (right). Replace these placeholders with actual photos/screenshots saved in `figures/`.

4.3 Observations

- The averaging and retry logic in `read_distance()` noticeably reduced spurious spikes in the point cloud compared to single-shot readings taken during early testing.
- Scan resolution was governed by `ROTATION_STEP_MAG` (angular resolution) and `D_VERTICAL/dh` (vertical resolution); tightening these increased point cloud density at the cost of scan time.
- The detection threshold (`DETECTED_THRESH_MM`) worked well for compact objects placed centrally on the plate, but was sensitive to the object's size relative to the sensor spacing – objects that did not occlude both sensors at once were under-sampled.
- Early-stopping on an empty ring correctly ended scans once the top of taller objects (like the pear) was reached, avoiding wasted time scanning past the object.

5 Conclusion

This project demonstrated a working 3D scanning pipeline built entirely from an Arduino, two ToF distance sensors, and three servos, with a lightweight Python toolchain to capture and visualize the resulting point clouds. The key embedded-systems challenge – multiplexing two identical I2C sensors with no built-in way to tell them apart – was solved cleanly using each sensor's XSHUT pin, and the scan routine's serpentine motion and early-stopping logic made full scans reasonably efficient.

Through this project, the main lessons learned were the importance of validating each hardware subsystem in isolation before integration, the need for defensive handling of noisy/unreliable sensor reads in embedded code, and how a very simple serial protocol (line-delimited JSON bracketed by start/end markers) was enough to decouple the embedded and host sides of a system cleanly.

Possible future improvements include:

- Replacing open-loop servo timing with closed-loop position feedback (e.g. rotary encoders) for more accurate and repeatable angular/elevation steps.
- Driving the rotation with a stepper motor or geared DC motor with an encoder instead of a servo, to allow continuous full-speed rotation rather than being limited to a servo's mechanical range.
- Fusing the two sensors' readings more tightly (e.g. triangulating a single 3D point per angle instead of two independent points) to improve accuracy on asymmetric objects.
- Post-processing the point cloud into a surface mesh (e.g. via Poisson reconstruction) instead of only visualizing raw points.
- Automatic exposure/threshold calibration at the start of a scan instead of a fixed `DETECTED_THRESH_MM`, to better support objects of varying size and reflectivity.